WP 09. Uncertainty-Aware Industrial Metrology and Tolerance Decisions

RESEARCH STATUS	Proposed study. No experimental results have been claimed.
CLUSTER	Precision Measurement and Industrial Quality
EXCEL CROSSWALK	Not specifically listed
CLOSEST PROJECT	paceval_Industrial_Piecewise_Calibration

Abstract

Metrology teams repeatedly apply parameterized measurement models with traceable uncertainty contributions and guard-banded decisions. This study defines a falsifiable and independently testable mathematical kernel for the paceval. Mathematical Engine. The workload and the evaluation criteria below are experimental plans rather than measurements.

1. Industrial scope and relationship to existing research

Distinctive contribution: Propagation of combined uncertainty and tolerance decisions, beyond calibration formula execution.

Workbook relationship: Not specifically listed. The workbook index and potential commercial benefit are subjective estimates, not measured commercial evidence.

2. Candidate mathematical kernels

Kernel 1

```
Measurand: y=a0+a1*x+a2*x*x+b*T
```

Kernel 2

```
Sensitivity: dy_dx=a1+2*a2*x
```

Kernel 3

```
First-order variance: u2=(dy_dx*ux)^2+(b*uT)^2+2*dy_dx*b*cov_xT
```

Kernel 4

```
Guardband margin: m=tol-abs(y-target)-k*u
```

Model expressions are mathematical specifications, not syntax-checked SDK strings. Implementation must declare units, parameter domains, formula IDs, regularization, model assumptions and reference tests.

3. How paceval is actually used

The host application owns parsing and dimensional validation of inputs, formula versioning, candidate generation, dataset provenance, graph-handle lifetime, independent solvers, aggregation and output. paceval owns every supported explicit scalar or composite mathematical expression and its repeated evaluation. Expression creation is performed once per stable model version. The host reuses each computation handle for all compatible input rows. Batch evaluation is used only where it preserves semantics and actually improves measured throughput. The selected paceval SDK or MCP interface must be identified in the experiment log; a local SDK call is not reported as a remote MCP benchmark.

C++ orchestrates candidate enumeration, fitted coefficients, domain-specific numerical solving, independent checking and I/O. paceval should own all supported scalar, composite cost, sensitivity and constraint formulas. Stable models use persistent computation handles and batch execution when beneficial.

4. Proposed numerical experiment and comparators

Evaluate 1e6 measurement samples with varying calibration revisions and 10 predefined covariance cases.

Baselines: Use high-precision Python mpmath and the NIST uncertainty propagation formulation as independent reference. Compare paceval to native C++ with the same assumptions.

Collect median, p95 and p99 latency across repeated runs; throughput, peak RAM, graph creation time, time per reused evaluation, energy where measured and complete pipeline wall time. Record hardware, compiler, precision, parallelism, batching, runtime version, warmup, data layout and input-sequence checksum. Use identical mathematical expressions, tolerances and unit handling in every baseline. Baseline alternatives should include optimized native C++ and, where domain-relevant, specialist solvers. Invalidate claims of engine-wide speedup if the measured advantage exists only in a small formula stage.

5. Independent validation and edge cases

1. Check covariance matrix positive semidefiniteness in the host.

2. Check analytic sensitivities with central differences away from branch points.

3. Propagate expanded uncertainty only after specifying coverage convention.

Predeclare the accepted absolute, relative and application-scaled errors. Verify each expression against an independent implementation using the same inputs and conventions. Include singular, limiting, invalid and near-boundary cases. If a formal interval claim is made, demonstrate coverage of an entire input interval using a method that actually bounds the interval; pointwise TINC checks alone are not such a proof.

6. Go or no-go assessment

A pilot is a technical go only if it passes the independently stated numerical and application acceptance tests and demonstrates either a repeatable improvement of at least 15% in full-pipeline latency or energy under matched conditions, or at least 25% reduction in measured formula-update/integration effort without a performance or quality regression. These are proposed decision thresholds, not measured outcomes. A result below the threshold is a no-go for the claimed benefit, even if the mathematical engine functions correctly.

Principal risk: First-order uncertainty propagation fails for some highly nonlinear or non-Gaussian regimes; run independent Monte Carlo checks.

7. Required release artifacts

Provide C++ sources, runtime version, formula manifest, dataset or synthetic generator with fixed seeds, native and high-precision references, failed-case logs, confidence intervals for timings and complete pipeline resource measurements. Never claim deployment, customer validation or certification without separate evidence.

8. Selected external references

1. https://www.nist.gov/publications/evaluating-expressing-and-propagating-measurement-uncertainty-nist-reference-materials

2. https://www.nist.gov/pml/nist-technical-note-1297/nist-tn-1297-5-combined-standard-uncertainty

paceval architecture: https://paceval.com/developers/

Context sources do not certify any paceval results.